

Module Documentation: XML Viewer and Navigation Subsystem

1. Overview

The `XMLPreview` component (`src/components/Plugins/XMLPreview.vue`) provides a Table of Contents (TOC) view for XML-based files as described in [SAS 7.2](#) and in the [BaSyx WebUI issue](#)..

The TOC is generated from the parsed XML document and shown alongside a formatted, syntax-highlighted XML content view. The TOC enables fast navigation, element selection, and synchronized scrolling between the tree and the text view.

This document describes how the TOC is created, which functions are responsible, how interaction between the tree and the XML content works, and maintenance considerations.

2. Purpose

The TOC implementation aims to:

- Present a concise, navigable hierarchical outline of an XML document's element tree.
- Provide element-level titles and line numbers for quick navigation.
- Allow selection/activation of tree nodes to scroll and highlight the corresponding XML lines.
- Support search highlighting and synchronized line-number display for context.

3. Responsibilities

Area	Description
TOC generation	Convert an XML DOM into a hierarchical <code>treeItems</code> structure with <code>id</code> , <code>title</code> , <code>lineNumber</code> , and <code>children</code> .
XML formatting	Produce a formatted textual representation of the XML that is split into lines for line-numbering and highlighting.
Navigation	Map tree activations to scrolling/highlighting in the formatted XML view.
Search synchronization	Support search across formatted lines, highlight results, and navigate between them.
UI layout	Render the TOC in a <code>v-treeview</code> next to the XML content with resizable tree panel.

4. Public API / Component Interface

`XMLPreview.vue` is a Vue component used as a plugin in the app. Relevant points of integration:

- Props:
 - `submodelElementData` (Object): expected to contain `.path` for the attachment to fetch.
- Exposed behavior (for consuming components):
 - The component shows a TOC and content view for the file referenced by `submodelElementData.path`.
 - No programmatic methods are exported; interactions are via UI (tree activation, search box, buttons).

5. Core TOC & View Functions

5.1 `buildXmlPresentation(xml: string): { text: string; tree: XmlTreeItem[] }`

- Entry point for transforming raw XML string into a formatted text (`text`) and a TOC tree (`tree`).
- Uses `DOMParser` to parse XML; if parsing fails, falls back to `fallbackFormatXml`.
- Adds the XML declaration (if present) as the first line and then serializes the document element via `serializeElement`.

Return value:

- `text`: the formatted XML as a newline-separated string.
- `tree`: array containing a single root `XmlTreeItem` (the document root) with nested `children`.

5.2 `serializeElement(element: Element, lines: string[], level: number, idPath = 'node'): XmlTreeItem`

- Recursively serializes a DOM element into `lines` while building a corresponding `XmlTreeItem`.
- For each element it:
 - Computes the current `lineNumber` as `lines.length + 1` (the first line the element starts on).
 - Builds an element `title` using `getDisplayTitle(element)` (uses `id` attribute when present).
 - Produces opening and closing tags, or condensed single-line entries for elements that have only text node children.
 - Appends serialized child nodes (elements, text, comments, CDATA) to `lines` and builds child `XmlTreeItems`.
- Returns an `XmlTreeItem` with `id`, `title`, `lineNumber`, and `children` if any.

Purpose:

- This function is the primary creator of the TOC structure and is what links tree nodes to exact line numbers in the formatted XML text.

5.3 `buildXmlView()`

- Calls `buildXmlPresentation()` and then stores:
 - `formattedXmlText` (raw formatted text used for searches and line counting),
 - `formattedXml` (HTML with Prism syntax highlighting when available),
 - `treeItems` (TOC tree used by `v-treeview`).
- Initializes `openedTreeNodes` and `activatedTreeNodes` so the tree opens to a shallow depth and the root is activated.
- Sets `selectedLineNumber` to the root's line number.

5.4 `handleTreeActivation(value: string[])` and `findTreeNodeById(...)`

- `handleTreeActivation` receives the activated node id(s) from the `v-treeview` and maps them to a `XmlTreeItem` via `findTreeNodeById`.
- On activation:
 - `selectedLineNumber` and `highlightedLineNumbers` are set to the node's `lineNumber`.
 - `scrollToLine(lineNumber)` is invoked to scroll the formatted view and the line-numbers column into center view.

5.5 Search and Navigation (`searchInXml`, `navigateSearchResult`)

- `searchInXml` scans `formattedXmlText` line-by-line for the search query and collects matching line numbers.
- On first result, it sets the active/highlighted line and scrolls to it.
- `navigateSearchResult` moves between found line numbers and scrolls/highlights accordingly.

6. Helper Functions

- `getXmlDeclaration(xml: string)` — extracts the XML declaration (e.g. `<?xml version="1.0"?>`) for inclusion as the first line.
- `buildAttributesString(element: Element)` — builds an escaped attribute string for display in the serialized opening tag.
- `getDisplayTitle(element: Element)` — computes a concise title for tree nodes, preferring an `id` attribute when available (`ElementName (id)`), otherwise the element name.
- `escapeXml` — escapes characters for safe HTML display in the serialized text.
- `serializeTextNode`, `serializeCommentNode`, `serializeCdataNode` — helpers to serialize non-element node types.
- `fallbackFormatXml` — a naive formatting fallback used when `DOMParser` fails; preserves readable indentation.
- `collectOpenNodeIds` — determines which nodes to open initially in the `v-treeview` (defaults to opening top-level children to depth 1).

7. Component State

Key reactive state variables used by the TOC and view:

- `xmlContent` (string): original XML string loaded from an attachment.
- `formattedXmlText` (string): the serialized, newline-separated text used for searching and line counting.
- `formattedXml` (string): HTML with Prism-highlighted markup used for rendering in the `pre/code` block.
- `treeItems` (`XmlTreeItem[]`): TOC model consumed by `v-treeview`.
- `openedTreeNodes` (string[]): action state for which tree nodes are expanded.
- `activatedTreeNodes` (string[]): currently activated tree node(s).
- `selectedLineNumber`, `highlightedLineNumbers` (numbers[]): UI highlighting and navigation helpers.
- `lineCount`, `lineNumberColumnWidth` (computed): used to render the line-number column.

8. Data and Control Flow

1. `initialize()` fetches the attachment blob (via `fetchAttachmentFile`) and sets `xmlContent`.
2. `watch(xmlContent)` triggers `buildXmlView()`.
3. `buildXmlView()` calls `buildXmlPresentation()` which:
 - Parses XML with `DOMParser`.
 - Calls `serializeElement()` recursively to build `lines` and `tree`.
4. The formatted `lines` array is joined into `formattedXmlText` and highlighted into `formattedXml`.
5. `treeItems` is set for the `v-treeview` and `openedTreeNodes/activatedTreeNodes` are initialized.
6. User interactions (tree activation, search, navigation) update `selectedLineNumber` and call `scrollToLine()` which synchronizes the content and line-number column scroll positions.

9. Use in the App

- The component is located at: `src/components/Plugins/XMLPreview.vue`.
- It is used wherever attachment previewing is needed (the component receives `submodelElementData.path` and fetches content via `useSMRepositoryClient`).
- Typical UI flows:
 - A user opens an AAS submodel element file attachment; `XMLPreview` loads and displays the TOC alongside the content.
 - Clicking a node in the TOC jumps to the element's starting line.
 - Searching the XML highlights result lines and allows stepping through matches.

10. Test Coverage

- There are no component-specific unit tests for `XMLPreview` in the repository as of this document.
- Suggested tests:
 - Parse a simple XML and assert `treeItems` contains expected nodes with correct `lineNumber`.
 - Activate a tree node and assert `selectedLineNumber` and `xmlContainer` scroll position change appropriately.
 - Provide malformed XML and assert `fallbackFormatXml` is used and `treeItems` is empty.

11. Technical Notes

- Parsing: uses `DOMParser` with MIME `application/xml` for namespace-aware parsing.
- Highlighting: uses `Prism` (client-side) to syntax-highlight XML content.
- Line mapping: `serializeElement` computes `lineNumber` based on `lines.length + 1` at the point the element's opening text is pushed — this is the anchor used for navigation.
- Tree IDs: generated as `${idPath}-${lineNumber}-${element.nodeName}` to be unique and stable for a given serialization.
- Layout: the TOC panel is resizable; a `ResizeObserver` keeps the tree body height in sync with the content wrapper.

12. Maintenance Notes

- Changing the serialization format (e.g., indentation rules or line-break behavior) will alter `lineNumber` anchors and invalidate TOC-to-line mappings. Keep `serializeElement` and any formatting logic in sync.
- If additional node types need representation (processing instructions, entity references), extend `serialize*` helpers and update `serializeElement` to include them.
- Be cautious about very large XML documents; the current in-memory serialization and Prism highlighting may be slow or memory-intensive for very large files.
- The `v-treeview` uses `item-value` set to `id`; avoid changing `id` scheme without migrating `openedTreeNodes/activatedTreeNodes` logic.

13. Known Risks

Risk	Impact	Recommendation
Line number desynchronization	If formatting changes, tree -> line mapping breaks	Keep serialization deterministic; add unit tests asserting line numbers for fixtures
Very large files	Performance/memory issues in the browser	Add streaming, pagination, or a size cutoff with a simplified viewer
Parser differences	Namespaced documents and non-standard XML may parse differently across browsers	Normalize parsing behavior and add test fixtures for namespace-heavy XML

14. Outlook

Possible future improvements:

- Add a settings option to collapse/expand the tree to arbitrary depths.
- Offer an option to limit highlighting and line-number rendering for very large files and provide a "light mode" TOC.
- Expose programmatic APIs (events or emitted messages) so parent components can react to node activation (e.g., for cross-component synchronization).
- Add unit/integration tests to validate TOC generation and navigation behavior.